

Expt No:

01

Date:

/ /2025

STUDY OF PLC & VERIFICATION OF LOGIC GATES

1. AIM:

To study the Programmable Logic controller (PLC) and verify its logical operations.

2. THEORY:

A programmable logic controller (PLC) is an industrial grade computer that is capable of being programmed to perform control functions. The programmable controller has eliminated much of the hardwiring associated with conventional relay control circuits. Other benefits include fast response, easy programming and installation, high control speed, network compatibility, troubleshooting and testing convenience, and high reliability.

The PLC is, then, basically a digital computer designed for use in machine control. Unlike a personal computer, it has been designed to operate in the industrial environment and is equipped with special input/output interfaces and a control programming language. Programmable controllers offer several advantages over a conventional relay type of control. Relays have to be hardwired to perform a specific function. When the system requirements change, the relay wiring has to be changed or modified. In extreme cases, such as in the auto industry, complete control panels had to be replaced since it was not economically feasible to rewire the old panels with each model changeover. The programmable controller has eliminated much of the hardwiring associated with conventional relay control circuits.

2.1. PARTS OF A PLC:

A typical PLC can be divided into parts, as illustrated in Figure 1. These are the central processing unit (CPU), the input/output (I/O) section, the power supply, and the programming device. There are two ways in which I/Os (Inputs/Outputs) are incorporated into the PLC: fixed and modular. Fixed I/O is typical of small PLCs that come in one package with no separate, removable units. The main advantage of this type of packaging is lower cost. The number of available I/O points varies and usually can be expanded by buying additional units of fixed I/O. One disadvantage of fixed I/O is its lack of flexibility.

Modular I/O is divided by compartments into which separate modules can be plugged. This feature greatly increases your options and the unit's flexibility. The power supply supplies DC power to other modules that plug into the rack. For large PLC systems, this power supply does not normally supply power to the field devices. With larger systems, power to field devices is provided by external alternating current (AC) or direct current (DC) supplies. For some small micro-PLC systems, the power supply may be used to power field devices. The processor (CPU) is the "brain" of the PLC. A typical processor usually consists of a microprocessor for implementing the logic and controlling the communications among the modules. The processor requires memory for storing user program instructions, numerical values, and I/O devices status.

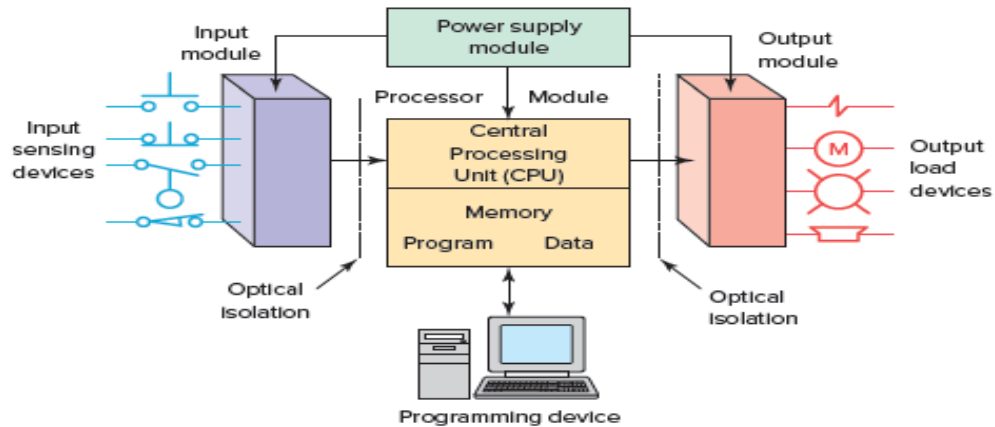


Figure 1. Typical parts of a programmable logic controller.

The I/O system forms the interface by which field devices are connected to the controller (Figure 1-14). The purpose of this interface is to condition the various signals received from or sent to external field devices. Input devices such as pushbuttons, limit switches, and sensors are hardwired to the input terminals. Output devices such as small motors, motor starters, solenoid valves, and indicator lights are hardwired to the output terminals. To electrically isolate the internal components from the input and output terminals, PLCs commonly employ an optical isolator, which uses light to couple the circuits together. The external devices are also referred to as “field” or “real-world” inputs and outputs.

The terms field or real world are used to distinguish actual external devices that exist and must be physically wired from the internal user program that duplicates the function of relays, timers, and counters. A programming device is used to enter the desired program into the memory of the processor. The program can be entered using relay ladder logic, which is one of the most popular programming languages. Instead of words, ladder logic programming language uses graphic symbols that show their intended outcome. A program in ladder logic is similar to a schematic for a relay control circuit. A personal computer (PC) is the most commonly used programming device. Most brands of PLCs have software available so that a PC can be used as the programming device. This software allows users to create, edit, document, store, and troubleshoot ladder logic programs.

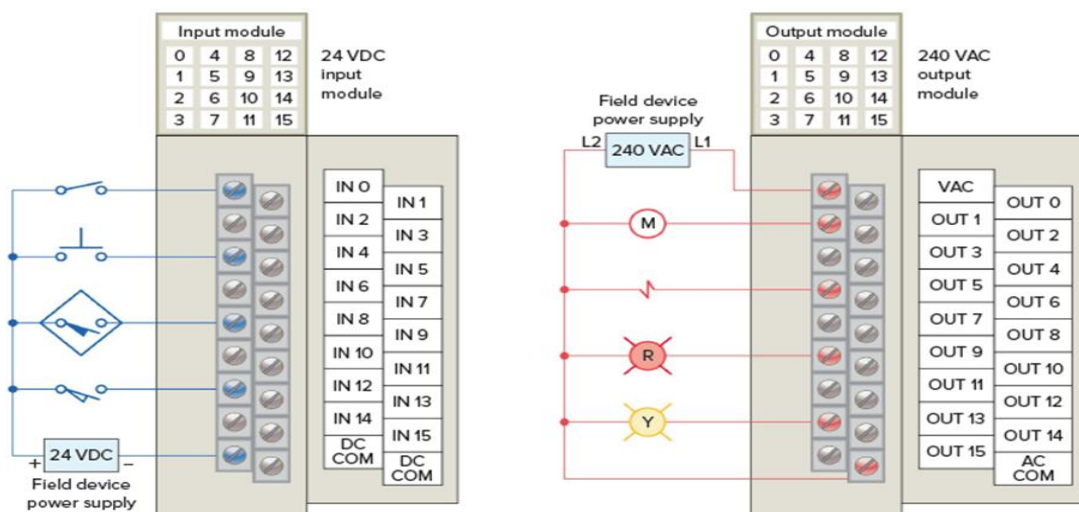


Figure 2. Typical PLC input/output (I/O) system connections.

2.2. PLC PROGRAMMING LANGUAGES:

The term PLC programming language refers to the method by which the user communicates information to the PLC.

- Ladder Diagram (LD) - a symbolic depiction of instructions arranged in rungs similar to ladder formatted schematic diagrams.
- Function Block Diagram (FBD) - a graphical depiction of process flow using simple and complex interconnecting blocks.
- Sequential Function Chart (SFC) - a graphical depiction of interconnecting steps, actions, and transitions.
- Instruction List (IL) - a low-level, text-based lang. that uses mnemonic instructions.
- Structured Text (ST) - a high-level, text-based language such as BASIC, C, or PASCAL specifically developed for industrial control applications. Ladder diagram language is the most commonly used PLC language and is designed to mimic relay logic. The ladder diagram is popular for those who prefer to define control actions in terms of relay contacts and coils, and other functions as block instructions.

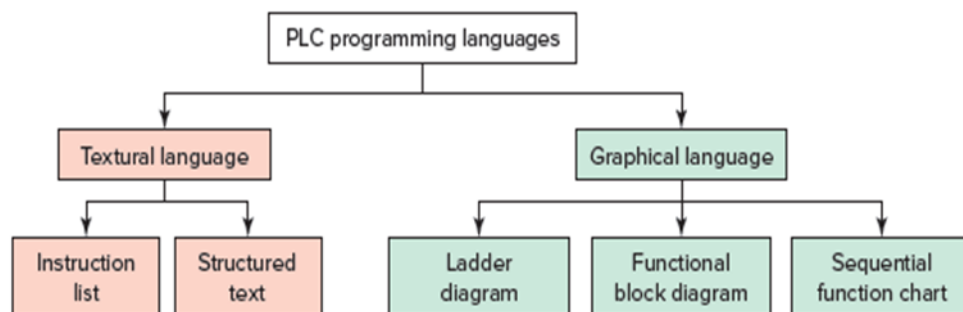
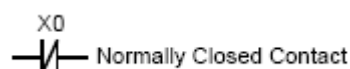


Figure 3. Languages associated with PLC programming.

There are three basic symbols used in ladder logic. The first one is NO - NC contacts: NO contact is an instruction that tells the processor to look at a specific bit in its RAM memory. If the bit is 1, the instruction is true and if it is 0, the instruction is false. The determining factor in choosing which bits in its memory to look at is the address. It could be some auxiliary bit (M), a timer contact (T), a counter contact (C), a state bit (S), or it might be connected to an external input (X).



NC contact plays the same role as the previous one, except that if the bit addressed is 1, the instruction is false and if it is 0, the instruction is true.



The second symbol is output: for outputting to the output module. If the instructions to the left on its rung have a true path to the leftmost vertical rail, then the PLC will set the bit to which it is referenced via the address to 1. If no complete true path is available, it will set the bit to 0.

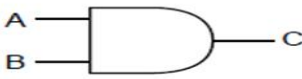
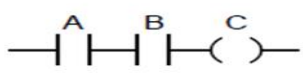
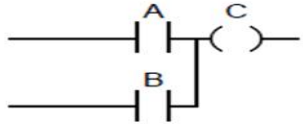
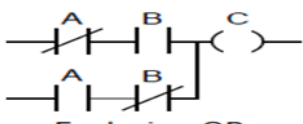
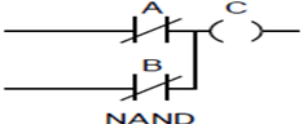
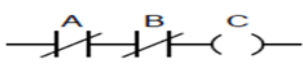


3. PROCEDURE (TO BE MODIFIED ACCORDING TO THE SOFTWARE):

- Open the PLC software and create a new project.
- Create the ladder logic diagram for the first logic gate (e.g., AND gate).
- Save the ladder logic program file.
- Connect the computer to the PLC hardware.
- Transfer the saved program from the computer to the PLC.
- Switch the PLC into RUN mode to activate the program.
- Provide inputs using the connected switches.
- Verify the output light matches the truth table for the gate you are testing.
- Clear the program from the PLC and repeat steps 2-8 for all other logic gates.
- This process completes the study of the PLC and the verification of its logical operations.

3.1. VERIFICATION OF LOGIC GATES:

EQUIVALENT LADDER/LOGIC DIAGRAMS

Logic Diagram	Truth Table	Ladder Diagram															
 AND Gate	<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>C</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </tbody> </table>	A	B	C	0	0	0	0	1	0	1	0	0	1	1	1	 AND Equivalent Circuit
A	B	C															
0	0	0															
0	1	0															
1	0	0															
1	1	1															
 OR Gate	<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>C</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </tbody> </table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	1	 OR Equivalent Circuit
A	B	C															
0	0	0															
0	1	1															
1	0	1															
1	1	1															
 Exclusive-OR Gate	<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>C</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	0	 Exclusive-OR Equivalent Circuit
A	B	C															
0	0	0															
0	1	1															
1	0	1															
1	1	0															
 NAND Gate	<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>C</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	C	0	0	1	0	1	1	1	0	1	1	1	0	 NAND Equivalent Circuit
A	B	C															
0	0	1															
0	1	1															
1	0	1															
1	1	0															
 NOR	<table border="1"> <thead> <tr> <th>A</th><th>B</th><th>C</th></tr> </thead> <tbody> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </tbody> </table>	A	B	C	0	0	1	0	1	0	1	0	0	1	1	0	 NOR
A	B	C															
0	0	1															
0	1	0															
1	0	0															
1	1	0															

4. **PROBLEM EXERCISES:**

- A simple circuit with one switch as a contact and one output as a coil. As the switch is opened or closed, the output goes on or off.

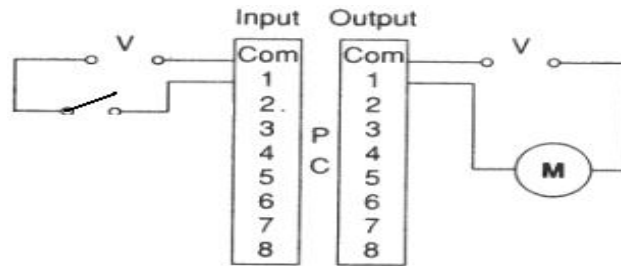


Figure 4.1.1. Connection Diagram.

Figure 4.1.2. Ladder Logic.

- When the start button is depressed, the coil energizes. When the button is released, the coil remains on.

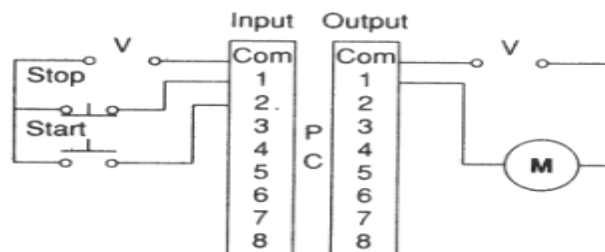


Figure 4.1.1. Connection Diagram.

Figure 4.1.2. Ladder Logic.

5. **RESULT:**

Thus, the Programmable Logic controller (PLC) has been studied and its various logic operations are verified.